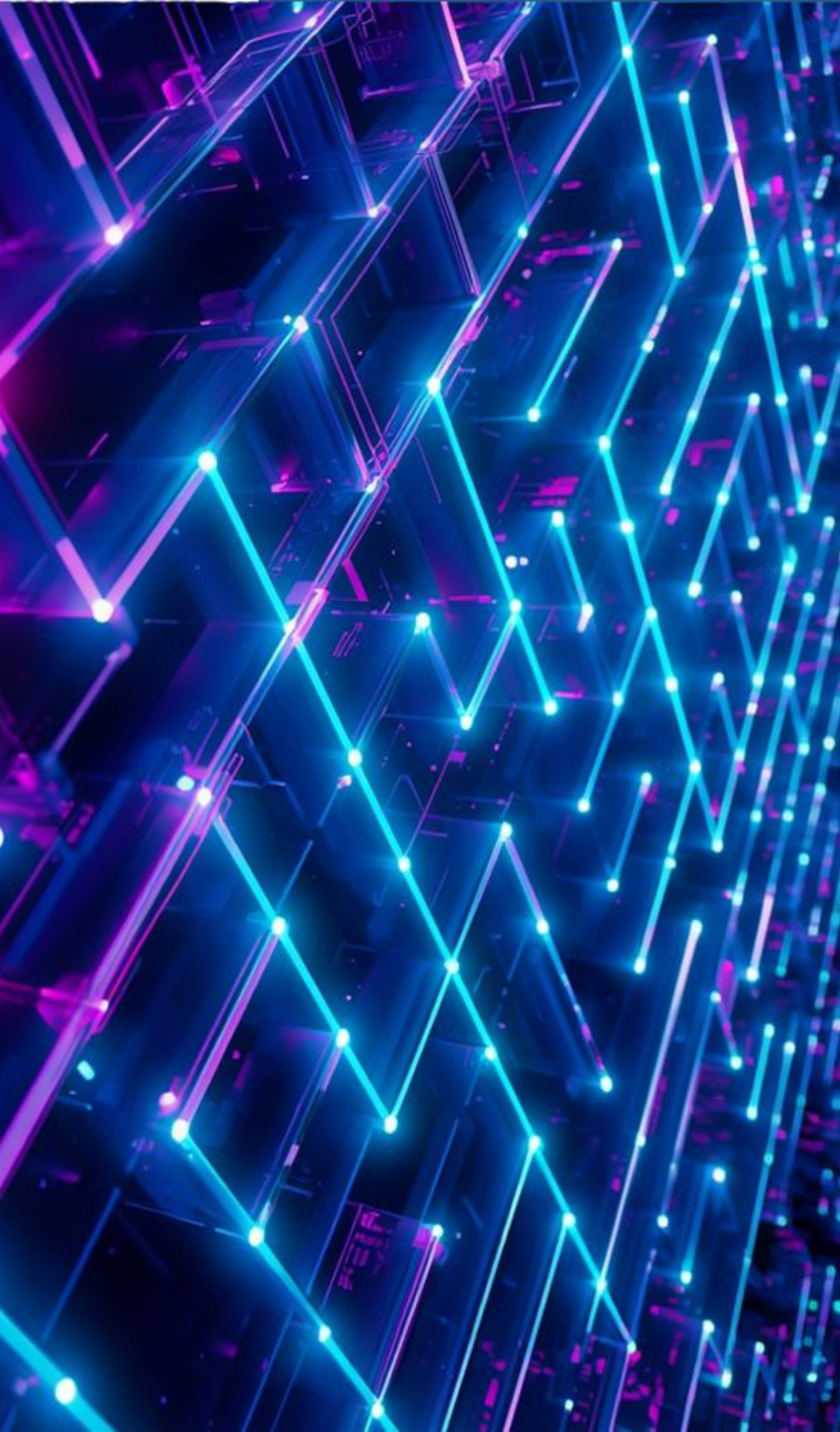




IAI 2026 PROJECT REPORT | TOPIC 4

Yıldız Technical University Mathematical Engineering

Zeynep Naz Tuzcu

Bahadır Ay

Görkem Yeşilyurt

Kübra Yeşilova

Mehmet Erdem

Eren Hasan Kasap

1. PROBLEM DEFINITION

General Description of the Problem In this project, within the scope of "The Shortest Path in Maze" problem, finding the shortest path from a starting point to a destination point in a two-dimensional maze is addressed. In accordance with the project definition, two different square or rectangular mazes have been designed: one medium-sized (at least 20x20) and one large (at least 50x50). It is aimed to find solutions for random starting and goal states in these mazes. The problem was solved using various blind and heuristic search algorithms, and the results were compared

```
typedef struct State {  
    int    row;    // row index  
    int    col;    // column index  
    float  h_n;    // heuristic value  
} State;
```



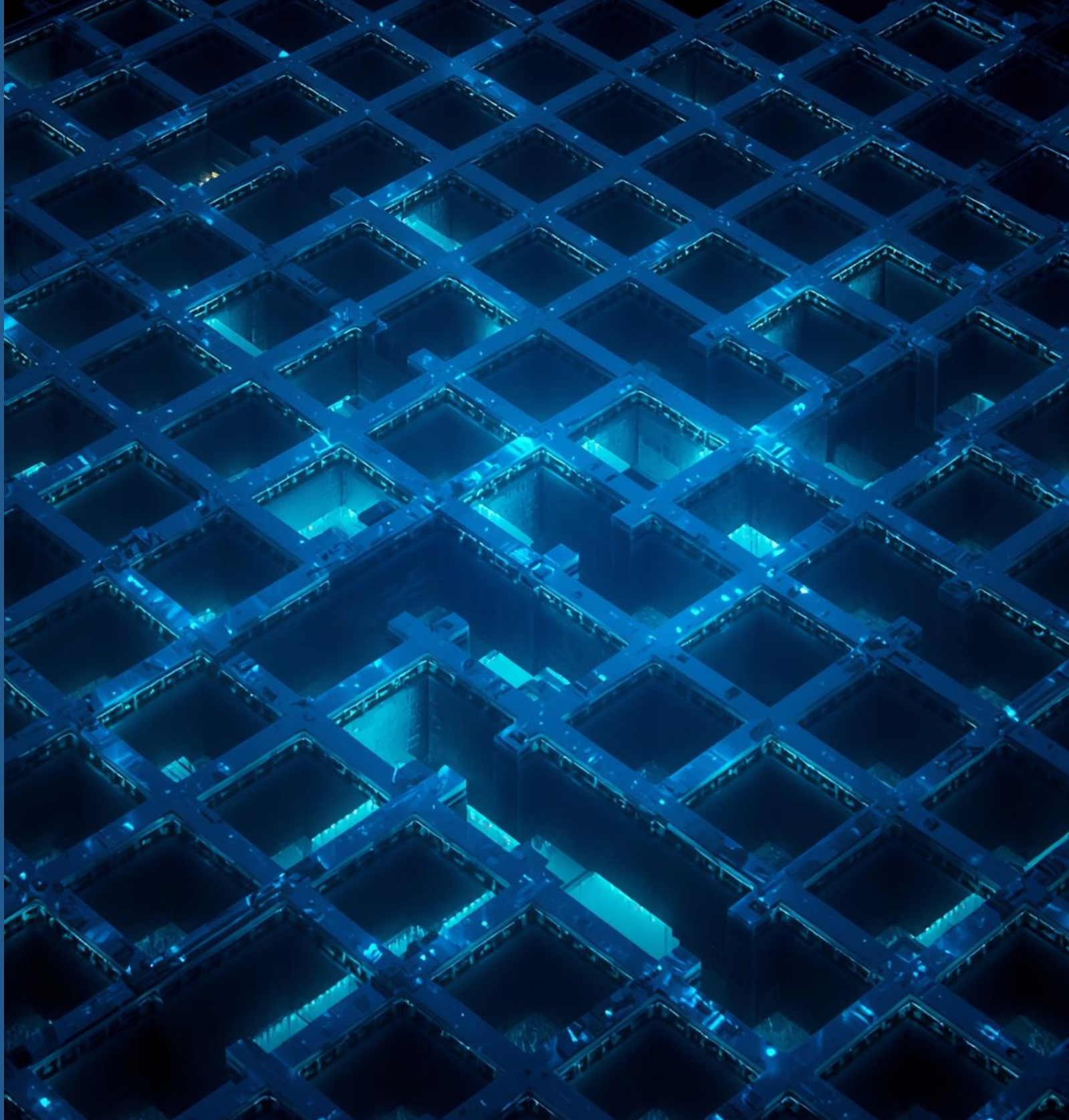

22X22 ARRAY STRUCT

Specifications:

- Size: 20×20 inner cells (22×22 array)
- Start Position: (1,1)
- Goal Position: (20,20)

Design Details:

- Equal distribution: 242 path cells and 242 wall cells
- Layout: Odd rows are traversable corridors; even rows are walls with strategic openings



52×52 ARRAY STRUCT

Specifications:

- Size: 50×50 inner cells (52×52 array)
- Start Position: (1,1)
- Goal Position: (48,48)

Design Characteristics:

- 1378 path cells vs 1326 walls
- Algorithmically generated with horizontal walls & vertical obstacles
- Designed to increase complexity via dead ends

HEURISTIC FUNCTION

Selected Heuristic: Manhattan Distance Manhattan Distance was used as the heuristic function. It is a standard and effective heuristic method for 4-way grid problems

Why Manhattan Distance?

- It is perfectly compatible with the 4-way movement structure of the maze (there is no diagonal movement).
- Since Euclidean distance assumes diagonal movement, it is not suitable for this problem.
- It is admissible: it never overestimates the actual cost.
- It is consistent: it satisfies the triangle inequality.

$$h(n) = |\text{row_current} - \text{row_goal}| + |\text{col_current} - \text{col_goal}|$$

PROOF OF ADMISSIBILITY

A heuristic $h(n)$ is admissible if $h(n) \leq h^*(n)$ is satisfied for every node n , where $h^*(n)$ is the actual minimum cost from n to the goal.

Manhattan Distance calculates the number of steps on a straight path, ignoring maze walls. The actual path is equal to or longer than this estimate due to walls. Therefore, $h(n) \leq h^*(n)$ is always satisfied

Numerical Example — Admissibility (20×20 Maze)

Node $n = (\text{row}=5, \text{col}=3)$. Goal state = $(\text{row}=18, \text{col}=18)$.

Heuristic estimate:

$$h(n) = |5 - 18| + |3 - 18| = 13 + 15 = 28$$

Actual shortest path $h^*(n)$:

Due to walls in the maze, the agent cannot travel in a straight line.

The real shortest path from $(5,3)$ to $(18,18)$ navigates around walls, so $h^*(n) \geq 28$.

Conclusion: $h(n) = 28 \leq h^*(n)$ ✓ The heuristic never overestimates. Admissibility confirmed.

PROOF OF CONSISTENCY

For consistency, $h(n) \leq c(n, a, n') + h(n')$ must hold for every node n and its successor n' reached via action a with cost c . In this maze, every step costs 1. Manhattan Distance decreases by at most 1 per step, so $h(n) \leq 1 + h(n')$ is always satisfied.

Numerical Example — Consistency (20×20 Maze)

Node $n = (5, 3)$. Successor $n' = (5, 4)$ via action `Move_Right` (cost = 1). Goal = $(18, 18)$.

$$h(n) = |5-18| + |3-18| = 13 + 15 = 28$$

$$h(n') = |5-18| + |4-18| = 13 + 14 = 27$$

$$c(n, \text{Move_Right}, n') = 1$$

Check: $h(n) \leq c(n, a, n') + h(n')$

$$28 \leq 1 + 27 = 28 \quad \checkmark$$

The inequality holds with equality because `Move_Right` reduces column distance by exactly 1, and Manhattan Distance tracks this perfectly. Since every action reduces Manhattan distance by at most 1, the consistency condition $h(n) \leq 1 + h(n')$ is always satisfied. ✓

COMPARISON OF ALTERNATIVE HEURISTICS

EUCLIDEAN VS. BFS (H(N)=0)

While the Euclidean distance estimates the shortest path by drawing a straight line, the case where $h(n) = 0$ behaves like BFS, traversing all nodes without using heuristic information. The A* algorithm uses the formula $f(n) = g(n) + h(n)$, where $h(n)$ determines the influence of the heuristic on the path.

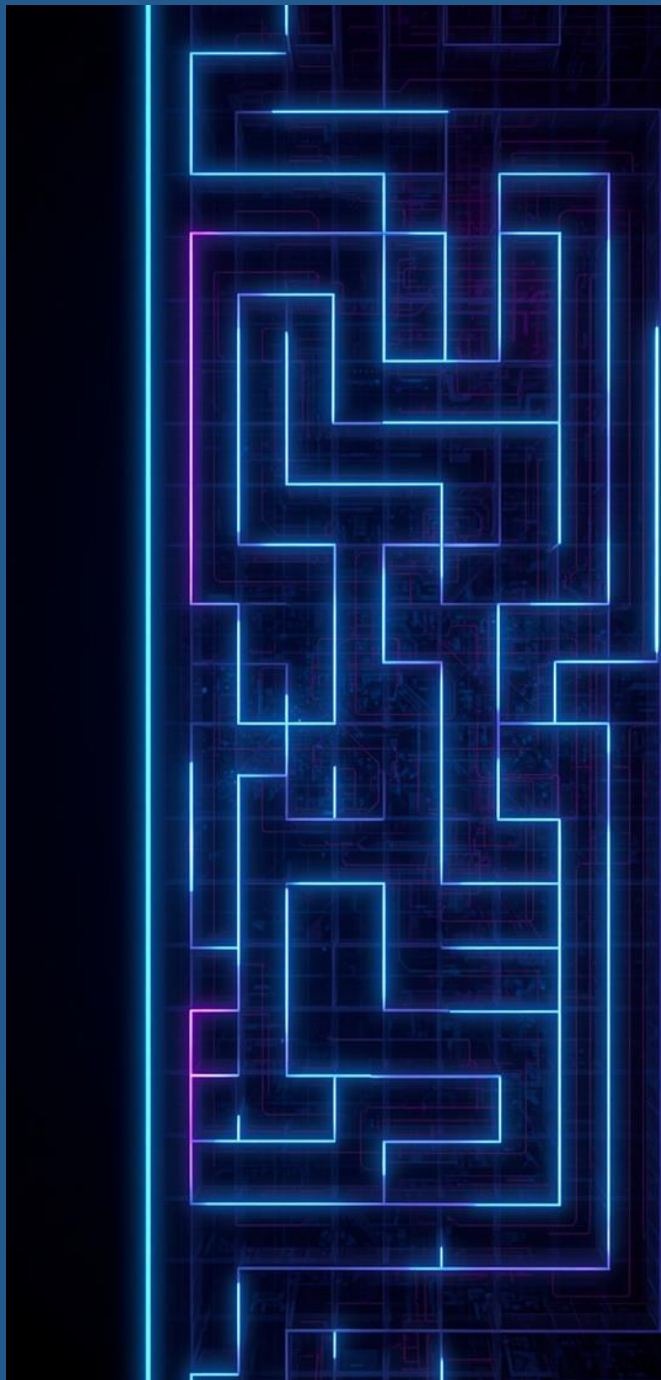
Heuristic	Admissible?	Consistent?	Efficiency
Manhattan Distance	Yes	Yes	High
Euclidean Distance	Yes*	Yes*	Medium (assumes diagonal)
$h(n) = 0$ (BFS)	Yes	Yes	Low (blind search)

A* PSEUDOCODE

```
function A_Star(start, goal):
    open_list = PriorityQueue()
    open_list.push(start, priority=0)
    closed_list = set()
    while open_list is not empty:
        current = open_list.pop()
    if current == goal: return reconstruct_path()
        closed_list.add(current)
        for neighbor in get_neighbors(current):
            if neighbor in closed_list: continue

    g_score = g_score[current] + cost(current,
                                        neighbor)
        if g_score < g_score[neighbor]:
            g_score[neighbor] = g_score
            f_score[neighbor] = g_score[neighbor] +
                heuristic(neighbor, goal)
            open_list.push(neighbor,
                priority=f_score[neighbor])
```

SIMULATION ENVIRONMENT AND TEST SCENARIOS



SIMULATION ENVIRONMENT

Hardware and
software environment:

- Operating System:
MS Windows, Apple
macOS, Linux (gcc ile
cross-platform)
- Programming
Language: C (gcc
compiler)
- Build Command: gcc
-o maze *.c -lm
- Hardware: Intel Core
i7, 16GB RAM

TEST SCENARIOS (S1-S3)

20x20 Mazes:

- S1: Standard maze
structure
- S2: Maze with narrow
corridors
 - S3: Maze with
predominantly open
areas
- Start and end points
vary.

TEST SCENARIOS (S4-S6)

50x50 Mazes:

- S4: Complex maze
structure
- S5: High wall density
- S6: Structures with
long corridors
 - The algorithmic
difficulty level has
been increased.

RESULT CHARTS

The results show that heuristic-based algorithms, especially Greedy and A*, significantly reduce computation time compared to uninformed search methods in larger maze scenarios.

Algorithm	S1 (ms)	S2 (ms)	S3 (ms)	S4 (ms)	S5 (ms)	S6 (ms)
BFS	10.5	8.0	6.7	51.8	141.3	144.1
Uniform Cost	9.2	8.5	7.8	52.1	149.7	136.3
DFS	7.5	9.8	8.9	89.9	104.8	111.0
Greedy (Manhattan)	4.8	3.8	3.9	10.4	75.1	85.0
A* (Manhattan)	7.5	6.8	8.8	34.0	73.0	70.5
Gen. A* ($\alpha=0.5$)	7.0	6.7	6.9	32.9	76.9	70.6
Gen. A* ($\alpha=0.0$)	9.0	8.0	8.0	47.9	138.8	137.5
Gen. A* ($\alpha=1.0$)	4.8	3.5	3.9	9.0	79.2	87.7

GENERATED NODES COMPARISON

Algorithm	S1	S2	S3	S4	S5	S6
BFS	543	471	471	1452	2825	2825
Uniform Cost	545	484	484	1465	2852	2852
DFS	417	473	441	1867	2073	2068
Greedy	222	125	146	365	1744	1811
A* (Manhattan)	431	376	392	1026	1893	1893
Gen.A* ($\alpha=0.5$)	431	376	392	1026	1893	1893
Gen.A* ($\alpha=0.0$)	545	484	484	1465	2852	2852
Gen.A* ($\alpha=1.0$)	222	126	147	367	1745	1812

OPTIMALITY

- BFS and Uniform Cost Search guaranteed the shortest path (optimal solution) on both mazes. Since $\text{step_cost}=1.0$ is constant, both yielded identical path costs (S1: 58, S4: 142).
- A* (Manhattan) and Gen. A* ($\alpha=0.0$ to $\alpha=0.5$) also returned optimal solutions; exactly the same cost values as BFS were obtained.
- DFS is not optimal: it produced paths up to 3 times longer than BFS on 20×20 (S2: 185 vs. 53), and 24% longer on 50×50 (S4: 182 vs. 142).
- Greedy and Gen. A* ($\alpha=1.0$) produced non-optimal results in some scenarios (S6: 226 vs. optimal 182).



EFFICIENCY — NODES GENERATED



For the 20×20 maze, Gen. A* ($\alpha=1.0$) and Greedy generated the fewest nodes (S1: 222 nodes). BFS generated the most (S1: 543 nodes).

A* with the Manhattan heuristic generated on average 21% fewer nodes than BFS on 20×20, and 35% fewer on 50×50.

When scaled to the 50×50 maze, BFS/UCS generated approximately 2825–2852 nodes, while A*/Gen.A*($\alpha=0.5$) remained between 1026–1893

OBSERVATIONS AND ANALYSIS

ALGORITHM BEHAVIORS

Upon examining optimal solutions and algorithm behaviors, it was observed that the A* algorithm demonstrates the most balanced performance in terms of both speed and computational cost. While BFS always finds the shortest path, it remains inefficient in terms of the number of nodes.

NON-OPTIMAL SITUATIONS

In DFS and Greedy searches, although the algorithm reaches the goal quickly, it has been observed that the paths found are not optimal (the shortest). In particular, the Greedy algorithm may choose inefficient routes due to obstacles in the maze by focusing excessively on heuristic values.

PARAMETER EFFECT

The α parameter in Generalized A* directly controls the evaluation function:
$$f(n) = (1-\alpha) \cdot g(n) + \alpha \cdot h(n)$$

When $\alpha=0 \rightarrow$ behaves like UCS (optimal, high node count).
When $\alpha=0.5 \rightarrow$ behaves like classical A* (optimal, balanced).
When $\alpha=1.0 \rightarrow$ behaves like Greedy (fast, non-optimal).
As α increases, speed improves at the cost of optimality.



SOURCE CODE EXPLANATIONS

File	Role
data_types.h	Global type definitions: State, Node, Queue, enums for ACTIONS and METHODS, Transition_Model.
GRAPH_SEARCH.h	Macro constants and declarations of all public functions used across the project.
GRAPH_SEARCH.c	Entry point (main). Reads user inputs, initialises the root node, dispatches to the chosen algorithm.
SpecificToProblem.c	All problem-specific logic: maze initialisation, state creation, transition model, heuristic, goal test, visualisation.
Standart_Search.c	Generic search infrastructure: frontier management, node expansion, tree traversal loops.
HashTable.c / .h	Open-addressing hash table used as the explored set to avoid revisiting states.

The program is built on the skeleton framework provided by the instructor and implements eight search algorithms:
Breadth-First Search (BFS),
Uniform-Cost Search (UCS),
Depth-First Search (DFS),
Depth-Limited Search (DLS),
Iterative Deepening Search (IDS),
Greedy Search,
A Search, and Generalised A* Search.*
Problem-specific implementations were developed in SpecificToProblem.c and HashTable.c/h, while the remaining files were kept unchanged in accordance with the project requirements.

MAZE AND HEURISTIC FUNCTIONS

BASIC FUNCTIONS

The ``Init_Maze()`` and ``Create_State()`` functions define the maze's boundaries and specify the start and goal states. The coordinates and movement costs of each cell are configured at this stage.

HEURISTIC AND TESTING

`Compute_Heuristic_Function()` determines the direction for A* by calculating the Manhattan distance. `Goal_Test()`, on the other hand, checks whether the current state matches the goal coordinates.

HYPOTHESES

HYPHOTESIS A

It is assumed that differences in algorithm performance will have a significant impact on the number of nodes and computation time, depending on the complexity of the maze and the use of heuristic functions in search strategies (BFS, DFS, A*, Greedy).

HYPHOTESIS B

When using appropriate heuristics such as the Manhattan distance, the A* and Generalized A* algorithms will reach optimal solutions much faster than basic search methods like BFS and DFS, while generating far fewer nodes.

HASH TABLE STRUCTURE

```
void Generate_HashTable_Key(const State *const state, unsigned char* key)
{
    sprintf((char*)key, "%d_%d", state->row, state->col);
    // e.g. state at row=3, col=7 -> key = "3_7"
}
```

```
void Generate_HashTable_Key(const State *const state, unsigned char* key)
{
    sprintf((char*)key, "%d_%d", state->row, state->col);
    // e.g. state at row=3, col=7 -> key = "3_7"
}
```

Configuration Constants (HashTable.h) •
HASH_TABLE_BASED_SIZE = 2503 — initial table capacity (prime number). •
HASH_TABLE_INCREASING_RATE = 70 — resizing threshold: when load factor exceeds 70%, the table doubles in size. • MAX_KEY_SIZE = 12 — maximum length of a key string.

Generate_HashTable_Key(state, key) The only problem-specific function in this file. Encodes a State as the string "row_col" using sprintf. Because (row, col) uniquely identifies a maze cell, this key guarantees that no two distinct states produce the same string.

CONCLUSION AND EVALUATION

PROJECT SUCCESS

- The project successfully modeled the shortest-path maze problem as a state-space search problem and implemented eight different search algorithms in C.
- Both uninformed and informed search strategies were tested on 20×20 and 50×50 maze scenarios.
- The experimental results matched the theoretical expectations of the algorithms.

A* PERFORMANCE

- A* with the Manhattan heuristic achieved the best balance between optimality and efficiency.
- It always found the optimal path while generating significantly fewer nodes than BFS and UCS.
- On large maze scenarios, A* reduced node expansion by approximately 35–40% compared to BFS.

Challenges and Solutions

- One major challenge was handling large search spaces efficiently in the 50×50 maze scenarios.
- This issue was addressed by using heuristic-based approaches such as Manhattan distance.
- Generalised A* was also tested with different α values to analyze the trade-off between speed and optimality.

FUTURE WORK

Although the current implementation already covers a representative set of search algorithms, several directions could extend the project further:

- Larger and more varied mazes: Performance can be measured on 100×100 or non-square mazes generated by classical procedures such as recursive backtracking or Prim's algorithm, in order to study how each algorithm scales as branching density and corridor length grow.
- Alternative heuristics: Chebyshev distance and diagonal distance can be implemented and compared, especially in variants of the problem that allow 8-directional movement. Weighted-A* variants ($f = g + w \cdot h$ with $w > 1$) could also be benchmarked to quantify the speed-versus-optimality trade-off more finely than the current α sweep.
- Bidirectional A*: Searching simultaneously from both the start and the goal and meeting in the middle is expected to reduce the number of expanded nodes substantially on long-corridor mazes such as Maze 2, and would be a natural next step.
- Non-uniform terrain costs: Extending the maze from binary walls to weighted terrain (plains, hills, mud, etc.) — which corresponds to Topic 5 in the project list — would make UCS and A* differ in path cost, providing a richer experimental setting.
- Empirical complexity analysis: Running each algorithm over many randomly generated start–goal pairs and reporting averaged generated-node counts, memory footprint, and wall-clock time would allow tighter empirical confidence intervals to be reported instead of single-run measurements.

REFERENCES

- [1] S. J. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, 4th ed., Pearson, 2020.
- [2] Course notes — Introduction to Artificial Intelligence, Yıldız Technical University, Department of Computer Engineering, Spring 2026.
- [3] Project specification — IAI 2026 Project: Topic 4, "The ShortestPath in Maze".
- [4] Wikipedia, "A* search algorithm", https://en.wikipedia.org/wiki/A*_search_algorithm
- [5] Wikipedia, "Maze generation algorithm", https://en.wikipedia.org/wiki/Maze_generation_algorithm
- [6] Wikipedia, "Manhattan distance", https://en.wikipedia.org/wiki/Taxicab_geometry
- [7] R. Sedgewick and K. Wayne, Algorithms, 4th ed., Addison-Wesley, 2011 — chapters on graph search and priority queues.
- [8] Math Is Fun, "Mazes", <https://www.mathsisfun.com/measure/mazes.html> (reference page cited in the project specification).

THANK YOU!
WE LOOK FORWARD TO YOUR QUESTIONS.